Konsol Tabanlı Metin Editörü (Mini Notepad++): Veri Yapıları ve Algoritmalar Odaklı Tasarım ve Gerçekleme

Burak Kılcı
Bilgisayar Mühendisliği Bölümü
Kocaeli Üniversitesi, Kocaeli, Türkiye
Öğrenci No: 230202111
230202111@kocaeli.edu.tr

***Özet*—Bu çalışmada, Notepad++ uygulamasının temel özelliklerini konsol ortamına taşıyan, fare kullanmadan yalnızca klavye kısayollarıyla yönetilen bir metin editörü tasarlanmış ve Python dili ile gerçeklenmiştir. Uygulama; dosya açma, kaydetme ve farklı kaydetme işlemleri için konsol içi bir dosya yöneticisi, karakter ve kelime bazlı seçim, kopyalama/kesme/yapıştırma, işlem tabanlı geri alma (undo), Knuth-Morris-Pratt (KMP) algoritması ile arama, eşleşmeler arasında gezinme ve toplu değiştirme özelliklerini sunmaktadır. Metin, her satırı bir karakter listesi olan dinamik bir satır listesinde tutulmuş; geri alma için özgün bir yığın (stack) yapısı tasarlanmış; seçim ise doğrusal başlangıç ve bitiş indeksleriyle izlenmiştir. Konsol erişimi harici bir kütüphane kullanılmadan Windows konsol API'si ve ANSI/VT kaçış dizileri üzerinden sağlanmıştır. Yapılan ölçümlerde satır listesi yapısının 50.000 satırlık bir dosyada ekleme işlemini tek parça metne göre yaklaşık 2400 kat hızlandırdığı, KMP'nin en kötü durum girdisinde naif aramadan yaklaşık 18 kat hızlı çalıştığı ve işlem tabanlı geri almanın 1000 düzenleme için anlık görüntü yöntemine kıyasla bellek kullanımını 4 büyüklük mertebesine varan oranda azalttığı gösterilmiştir.**

***Anahtar Kelimeler*—metin editörü, konsol uygulaması, veri yapıları, KMP algoritması, geri alma yığını, Windows konsol API**

Abstract*—*In this work, a keyboard-only, console-based text editor inspired by Notepad++ was designed and implemented in Python. The editor provides an in-console file manager, character- and word-level selection, clipboard operations, an operation-based undo stack, KMP-based search with match navigation and replace-all. Text is stored as a dynamic list of character lists, and the console is driven directly through the Windows Console API and ANSI/VT escape sequences without any external GUI library. Experiments show substantial gains over naive alternatives in insertion cost, search time and undo memory usage.

I. Giriş

Metin editörleri, yazılım geliştirme sürecinin en temel araçlarındandır. 1970'lerde geliştirilen `vi` ve Emacs gibi editörler grafik arayüz bulunmayan terminallerde çalışacak şekilde tasarlanmış, bu nedenle tüm işlemleri klavye üzerinden yürütmüştür. Günümüzde Notepad++ [12] gibi grafik arayüzlü editörler yaygınlaşmış olsa da, sunucu yönetimi, gömülü sistemler ve uzak bağlantılar gibi pek çok senaryoda konsol tabanlı editörler hâlâ vazgeçilmezdir.

Konsol tabanlı bir editör geliştirmek, grafik arayüzlü bir uygulamaya göre farklı zorluklar içerir. Konsol, varsayılan olarak satır tamponlu (line-buffered) çalışır; yani kullanıcı Enter tuşuna basana kadar programa hiçbir girdi ulaşmaz. Ayrıca `Ctrl+Shift+Sol Ok` gibi tuş kombinasyonları standart giriş akışında ayırt edilemez, renkler ve imleç şekli ise özel kaçış dizileriyle kontrol edilmelidir. Bunlara ek olarak metnin bellekte nasıl tutulacağı, düzenleme işlemlerinin maliyeti, geri alma geçmişinin bellek kullanımı ve arama algoritmasının verimliliği gibi veri yapısı ve algoritma kararları uygulamanın başarımını doğrudan belirler.

Bu projenin temel amaçları şunlardır: (i) konsol tabanlı uygulama geliştirme pratiği kazanmak, (ii) dosya sistemi üzerinde gezinme ve dosya yönetimi işlemlerini gerçekleştirmek ve (iii) metin işleme, arama, değiştirme ve düzenleme algoritmalarını uygulamak. Bu doğrultuda geliştirilen uygulamanın katkıları aşağıda özetlenmiştir:

- Harici bir kütüphane kullanılmadan Windows konsol API'si üzerinden ham (raw) klavye girişi okunmuş; Ctrl, Shift ve AltGr kombinasyonları ayırt edilmiştir.
- Metin için her satırı bir karakter listesi olan iki seviyeli dinamik bir veri yapısı kullanılmış ve alternatif yapılarla karşılaştırmalı olarak gerekçelendirilmiştir.
- Metnin kopyası yerine yalnızca yapılan işlemi saklayan, kelime bazlı birleştirme ve toplu işlem desteği olan bir geri alma yığını tasarlanmıştır.
- Arama için KMP algoritması gerçeklenmiş, çakışan eşleşmeler elenerek değiştirme işlemiyle tutarlılık sağlanmıştır.
- Uygulama otomatik testlerle doğrulanmış ve tüm bileşenlerin başarımı deneysel olarak ölçülmüştür.

Raporun geri kalanı şu şekilde düzenlenmiştir: Bölüm II sistemin mimarisini, veri yapılarını ve algoritmalarını; Bölüm III test yöntemini, ekran görüntülerini ve başarım ölçümlerini; Bölüm IV ise sonuçları ve gelecek çalışmaları sunmaktadır.

Tablo I
Aday Programlama Dillerinin Karşılaştırılması

Ölçüt	C	Java	Python
Ham klavye girişi	Var (WinAPI)	Yok*	Var (`ctypes`)
Ctrl+Shift+Yön ayrımı	Var	Yok*	Var
Dinamik dizi/liste	Elle	Hazır	Hazır
Unicode metin	Zahmetli	Hazır	Hazır
`main.exe` üretimi	Derleyici	Ek araç	PyInstaller
Kod uzunluğu (tahmini)	Uzun	Orta	Kısa

*JNI/JNA gibi yerel kod köprüleri olmadan mümkün değildir.

II. Yöntem

A. Gereksinim Analizi ve Programlama Dili Seçimi

Proje tanımında programlama dili serbest bırakılmış, ancak uygulamanın konsolda çalışması, farenin kullanılmaması, tüm işlemlerin klavye kısayollarıyla yapılması, Türkçe karakter desteği, renkli vurgulama ve `main.exe` ile başlatılması istenmiştir. Bu gereksinimlerin en kritik olanı, `Ctrl+Yön` ile `Ctrl+Shift+Yön` gibi kombinasyonların birbirinden ayırt edilmesidir. Aday diller bu açıdan Tablo I üzerinde karşılaştırılmıştır.

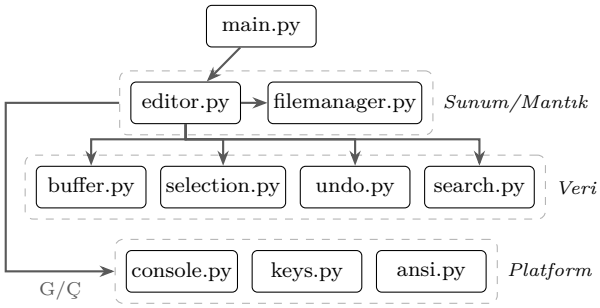
Java'nın standart kütüphanesi konsolu ham moda alamadığı için elenmiştir. C dili gerekli tüm erişimi sağlasa da dinamik bellek yönetimi ve Unicode işleme kodu önemli ölçüde uzatmaktadır. Python ise standart kütüphanesindeki `ctypes` modülü [10] sayesinde Windows API fonksiyonlarını doğrudan çağırabildiği, hazır dinamik listeleri ve yerleşik Unicode desteği bulunduğu için seçilmiştir. Python kodu PyInstaller [11] aracıyla tek dosyalık `main.exe` hâline getirilmektedir. Proje tanımındaki "harici GUI kütüphanesi kullanılmayacaktır" şartı gereği uygulamada yalnızca Python standart kütüphanesi kullanılmıştır.

B. Sistem Mimarisi

Uygulama, sorumlulukların ayrılması (separation of concerns) ilkesine göre dört katmana ve on modüle ayrılmıştır (Şekil 1, Tablo II). Platform katmanı işletim sistemine bağımlı tek katmandır; üst katmanlar konsolu yalnızca `read_key`, `write`, `size` ve pano fonksiyonlarından oluşan küçük bir arayüz üzerinden kullanır. Editör nesnesi konsol nesnesini dışarıdan parametre olarak alır (bağımlılık enjeksiyonu). Bu tasarım, Bölüm III'te açıklanacağı gibi gerçek konsol yerine sahte bir konsol verilerek tüm uygulamanın otomatik olarak test edilmesini mümkün kılmıştır.

C. Konsol Giriş/Çıkış Katmanı

1) Ham giriş: Python'un `input()` fonksiyonu Enter beklediği ve değiştirici tuşları raporlamadığı için kullanılamaz. Bunun yerine konsol `SetConsoleMode` [9] ile satır tamponu (`ENABLE_LINE_INPUT`), yankı (`ENABLE_ECHO_INPUT`), `Ctrl+C` sinyali (`ENABLE_PROCESSED_INPUT`), fare girişi ve fare ile hızlı düzenleme (`QUICK_EDIT`) kapatılarak ham moda alınmıştır. Fare girişinin kapatılması projenin "fare



Şekil 1. Uygulamanın katmanlı mimarisi ve modüller arası bağımlılıklar.

Tablo II
Modüller ve Sorumlulukları

Modül	Sorumluluk	Satır
`console.py`	Ham giriş, VT modu, tuş çözümleme, pano	293
`keys.py`	Tuş olayı sınıfı (`Key`)	24
`ansi.py`	Renk kodları, konumlandırma, hizalama	50
`buffer.py`	Metin veri yapısı (`TextBuffer`)	121
`selection.py`	Seçimin indekslerle takibi	39
`undo.py`	`Stack` ve `Action` sınıfları	59
`search.py`	KMP ve tüm eşleşmeleri bulma	67
`filemanager.py`	Konsol içi dosya yöneticisi	233
`editor.py`	Komutlar, tuş yönlendirme, çizim	748
`main.py`	Giriş noktası, hata yakalama	31
Toplam		**1665**

kullanılmayacaktır" şartını işletim sistemi seviyesinde garanti eder. Ardından her tuş olayı `ReadConsoleInputW` [8] ile bir `INPUT_RECORD` yapısı olarak okunur. Bu yapı, sanal tuş kodu (VK), donanım tarama kodu, Unicode karakter ve Ctrl/Alt/Shift durum bayraklarını içerir. Yapıların Python tarafındaki karşılıkları `ctypes.Structure` ile tanımlanmış; `KEY_EVENT_RECORD` için 16, `INPUT_RECORD` için 20 baytlık Windows bellek düzeninin birebir sağlandığı doğrulanmıştır.

2) Tuş çözümleme: Ham olaylar Tablo III üzerindeki kurallarla platformdan bağımsız `Key` nesnelerine dönüştürülür. Burada iki özel durum önemlidir. Birincisi, Türkçe Q klavyede `@ # { } [ ] \ |` karakterleri AltGr tuşuyla yazılır ve Windows AltGr'yi "Ctrl+Alt" olarak bildirir. Ctrl'ye basılı her olay kısayol sayılsaydı bu karakterler yazılamazdı; bu nedenle Ctrl ile birlikte Alt da basılıysa olay normal karakter olarak yorumlanır. İkincisi, ASCII tablosunda `Ctrl+H` ile Backspace aynı koda (8) karşılık gelir ve bazı terminal öykünücüleri `Ctrl+H`'yi Backspace olarak iletir. Bu durum, H tuşunun tarama kodu (0x23) ve klasik konsolda gerçek `Ctrl+Backspace`'in `0x7F` karakteri üretmesi kullanılarak ayırt edilmiştir.

3) Çıkış ve VT dizileri: Renkler ve imleç kontrolü için çıkış tamponunda `ENABLE_VIRTUAL_TERMINAL_PROCESSING` bayrağı açılarak ECMA-48 standardındaki ANSI kaçış dizileri kullanılmıştır [6], [7]. Kullanılan diziler Tablo IV üzerinde verilmiştir. İmlecin proje tanımında istenen "yanıp

Tablo III
Tuş Olaylarının Çözümlenme Kuralları (Öncelik Sırasıyla)

#	Koşul	Sonuç
1	VK=Backspace ve (tarama=0x23 veya Ctrl+`\x08`)	`Ctrl+H`
2	VK özel tuş tablosunda	Yön, Enter, ESC, …
3	Yazılabilir karakter ve (Ctrl yok veya Alt var)	`CHAR` (AltGr dahil)
4	Ctrl basılı ve VK ∈ [A–Z]	`Ctrl+harf`
5	VK=0 ve karakter `\r` / `\t`	Enter / Tab (yapıştırma)
6	Diğer (yalnız Shift, Ctrl, …)	Yok sayılır

Tablo IV
Kullanılan ANSI/VT Kaçış Dizileri

Dizi	Görevi
`ESC[?1049h / l`	Alternatif ekran tamponuna geç / dön
`ESC[?7l`	Satır sonunda otomatik kaydırmayı kapat
`ESC[5 q`	İmleç: yanıp sönen dikey çizgi
`ESC]12;#ffffff BEL`	İmleç rengi: beyaz
`ESC[?25l / h`	Çizim sırasında imleci gizle / göster
`ESC[`s`;`k`H`	İmleci s. satır, k. sütuna taşı
`ESC[K`	Satırın kalanını temizle
`ESC[30;103m`	Seçim: sarı arka plan
`ESC[97;44m`	Arama eşleşmesi: mavi arka plan

sönen dikey çizgi" biçimi DECSCUSR dizisi (`ESC[5 q`) ile, beyaz rengi ise OSC 12 dizisi ile ayarlanır. Uygulama alternatif ekran tamponunda çalıştığından, çıkışta kullanıcının konsoldaki önceki içeriği bozulmadan geri gelir. Metin `WriteConsoleW` ile UTF-16 olarak yazıldığı ve kod sayfası 65001'e (UTF-8) ayarlandığı için Türkçe karakterler doğru görüntülenir.

D. Metin Veri Yapısı

Proje tanımı, metindeki her karakterin seçilen bir veri yapısında tutulmasını ve dosya içeriğinin dinamik olarak yönetilmesini istemektedir. Bu amaçla iki seviyeli bir yapı seçilmiştir: n satırlık bir metin $L = [\ell_0, \ell_1, \ldots, \ell_{n-1}]$ dinamik listesiyle, her satır da $\ell_i = [c_0, c_1, \ldots, c_{m_i-1}]$ karakter listesiyle temsil edilir. Satır sonu karakterleri saklanmaz; satırlar arası sınır liste yapısının kendisiyle ifade edilir. Böylece bir satırın numarası her zaman o satırın liste indeksinin bir fazlasıdır ve satır eklendiğinde ya da silindiğinde numaralar ek bir işlem gerekmeden kendiliğinden güncellenir.

Tüm düzenleme işlemleri yalnızca iki temel fonksiyondan geçer: `insert_text(r, c, metin)` ve `delete_range(başlangıç, bitiş)`. Çok satırlı ekleme sırasında imlecin sağında kalan kuyruk kesilir, metnin ilk parçası satıra eklenir, kalan parçalar yeni satırlar olarak araya yerleştirilir ve kuyruk son satırın sonuna eklenir. Silme işlemi ise ilk satırın başı ile son satırın kuyruğunu birleştirip aradaki satırları kaldırır. Silinen metin geri alma işlemi için çağırana döndürülür.

Tablo V
Metin Veri Yapılarının Asimptotik Karşılaştırması

Yapı	Erişim	Düzenleme	Zorluk
Tek parça dizi	$O(N)$	$O(N)$	Düşük
Karakter bağlı listesi	$O(N)$	$O(1)^{\dagger}$	Orta
Gap buffer [3]	$O(N)$	$O(1)^{\ddagger}$	Orta
Piece table [4]	$O(p)$	$O(p)$	Yüksek
Satır listesi (seçilen)	$\mathbf{O(1)}$	$\mathbf{O(m)}$	**Düşük**

Erişim: (r, c) konumuna ulaşma. N: toplam karakter, m: satır uzunluğu, p: parça sayısı. †Konum bilindiğinde. ‡Amortize, boşluk imleçteyken.

Tablo V, seçilen yapıyı yaygın alternatiflerle karşılaştırmaktadır. Tek parça bir dizide dosyanın ortasına yapılan her ekleme, ekleme noktasından sonraki tüm karakterlerin kaydırılmasını gerektirir. Karakterlerin bağlı listede tutulması eklemeyi sabit zamana indirse de, (r, c) konumuna ulaşmak için listenin baştan dolaşılması gerekir; ekran çizimi ve imleç hareketi gibi en sık yapılan işlemler bu nedenle yavaşlar. Gap buffer ve piece table ise yüksek başarımlı editörlerde kullanılan ancak gerçeklemesi daha karmaşık yapılardır. Bir metin editöründe düzenlemeler neredeyse her zaman tek bir satırda yoğunlaştığı ve satırlar genellikle kısa olduğu için ($m \ll N$), satır listesi yapısı hem $O(1)$ erişim hem de $O(m)$ düzenleme maliyetiyle bu proje için en dengeli seçenektir. Bu öngörü Bölüm III'te deneysel olarak doğrulanmıştır.

E. Seçim ve İndeks Dönüşümü

Proje tanımı gereği seçili metin, başlangıç ve bitiş indeksleriyle takip edilir. `Selection` sınıfında `start` seçimin başladığı sabit indeksi, `end` ise imlecin hareketli indeksini tutar. Kullanıcı sola doğru seçim yaptığında $end < start$ olabileceğinden, kullanım anında $[\min, \max)$ aralığı hesaplanır. Doğrusal indeks ile (r, c) konumu arasındaki dönüşüm, her satır sonundaki ayırıcı da bir karakter sayılarak (1) ile yapılır:

$$\mathrm{idx}(r, c) = \sum_{k=0}^{r-1} (|\ell_k| + 1) + c \tag{1}$$

Ters dönüşüm, indeksten satır uzunluklarını sırayla çıkararak yapılır. `Ctrl+Yön` karakter karakter, `Ctrl+Shift+Yön` ise kelime kelime seçim yapar. Kelime sınırları karakterler üç sınıfa ayrılarak belirlenir: kelime karakteri (harf, rakam, alt çizgi; Türkçe harfler `isalnum()` ile doğru tanınır), boşluk ve noktalama. Sağa kelime hareketi önce aynı sınıftaki ardışık karakterleri, sonra boşlukları atlar; sola hareket bunun tersini uygular. Satır başında ya da sonunda hareket bir önceki ya da sonraki satıra geçer. Seçim aktifken yazılan bir karakter, Backspace ya da yapıştırma işlemi önce seçili metni siler.

F. Geri Alma (Undo) Mekanizması

Geri alma için en basit yöntem her düzenlemeden önce metnin tam bir kopyasını (anlık görüntü) saklamaktır;

Algoritma 1 Geri Alma

```
yordam GeriAl
    a ← Yığın.Pop
    eğer a = Boş ise
        döndür "Geri alınacak işlem yok"
    koşul sonu
    TersUygula(a)
    İmleci a.imleç_önce konumuna taşı; seçimi temizle
yordam sonu
yordam TersUygula(a)
    eğer a.tür = Insert ise
        Sil(a.p, Son(a.p, a.t))
    değilse eğer a.tür = Delete ise
        Ekle(a.p, a.t)
    değilse                                   ▷ Batch
        her i ← k down to 1 yap
            TersUygula(a.alt_i)
        döngü sonu
    koşul sonu
yordam sonu
```

ancak bu yöntemin bellek maliyeti $O(N \cdot k)$'dir (k: düzenleme sayısı). Bu projede bunun yerine Komut (Command) tasarım deseninden [5] esinlenen işlem tabanlı bir yaklaşım kullanılmıştır. Yığına yalnızca yapılan işlem kaydedilir ve geri alma işlemi bu kaydın tersini uygular:

- `INSERT`(p, t): p konumuna t eklendi. Tersi: p ile son(p, t) arası silinir.
- `DELETE`(p, t): p konumundan t silindi. Tersi: t, p konumuna eklenir.
- `BATCH`$(a_1, \ldots, a_k)$: Toplu işlem. Tersi: alt işlemler $a_k, \ldots, a_1$ sırasıyla geri alınır.

Yığın, Python listesi üzerinde `push`, `pop`, `peek`, `is_empty` ve `size` işlemlerine sahip özgün bir `Stack` sınıfı olarak tasarlanmıştır; tüm işlemler $O(1)$'dir. Yığın 1000 kayıtla sınırlandırılmış, sınır aşıldığında en eski kayıt atılmaktadır. Her karakter için ayrı bir kayıt tutulması kullanıcının bir kelimeyi silmek için defalarca `Ctrl+Z`'ye basmasına yol açacağından, ardışık ve bitişik karakter eklemeleri tek bir `INSERT` kaydında birleştirilir. Boşluk karakteri, imleç hareketi, silme ya da kaydetme işlemi birleştirmeyi sonlandırır; böylece geri alma kelime kelime çalışır.

Dosyanın kaydedilip kaydedilmediği de geri alma yığını üzerinden belirlenir. Her `Action` nesnesine benzersiz bir numara verilir; kaydetme anında yığının tepesindeki kaydın numarası saklanır. Dosya, ancak yığının o anki tepe numarası bu değerden farklıysa "değiştirilmiş" sayılır. Bu yöntem sayesinde kullanıcı kayıttan sonra bir değişiklik yapıp geri aldığında, çıkışta gereksiz yere "Kaydetmek istiyor musunuz?" sorusu sorulmaz. Geri alma algoritması Algoritma 1 üzerinde verilmiştir.

Algoritma 2 KMP ile Tüm Eşleşmeleri Bulma

Girdi: Metin $T[0..n-1]$, kelime $P[0..m-1]$
Çıktı: Eşleşmelerin başlangıç indeksleri

```
lps ← LPSOluştur(P);  S ← ∅
i ← 0;  j ← 0
iken i < n yap
    eğer T[i] = P[j] ise
        i ← i + 1;  j ← j + 1
        eğer j = m ise
            S ← S ∪ {i − j};  j ← lps[j − 1]
        koşul sonu
    değilse eğer j > 0 ise
        j ← lps[j − 1]                  ▷ i geri gitmez
    değilse
        i ← i + 1
    koşul sonu
döngü sonu
döndür S
```

G. Arama: Knuth-Morris-Pratt Algoritması

Proje tanımı, arama işlemlerinin bir string eşleşme algoritmasıyla yapılmasını istemektedir. Naif arama, bir uyuşmazlıkta metinde geri dönerek bir sonraki konumdan yeniden başladığı için en kötü durumda $O(n \cdot m)$ zaman alır (n: metin, m: aranan kelime uzunluğu). KMP algoritması [1], [2] ise aranan kelimenin kendi içindeki tekrarları önceden LPS (*Longest Proper Prefix which is also Suffix*) tablosunda hesaplar. lps$[i]$, kelimenin ilk $i{+}1$ karakterinde hem önek hem sonek olan en uzun özel parçanın uzunluğudur. Bir uyuşmazlık olduğunda metindeki konum hiç geri alınmaz; yalnızca kelimedeki konum lps değerine çekilir. Böylece toplam süre $O(n + m)$ olur. Örneğin "ABABCABAB" kelimesi için LPS tablosu $[0, 0, 1, 2, 0, 1, 2, 3, 4]$ şeklindedir. Algoritma 2, gerçeklenen arama yordamını göstermektedir.

KMP doğası gereği çakışan eşleşmeleri de bulur ("aaaa" içinde "aa" için 0, 1 ve 2). Değiştirme işleminin tutarlı olması için `find_all` fonksiyonu her satırda eşleşmeleri soldan sağa tarar ve bir önceki eşleşmenin bitişinden önce başlayanları eler. Aranan kelimeler satır sınırını aşamayacağından arama her satıra ayrı ayrı uygulanır; sonuç, (r, c) sırasına göre sıralı bir liste olarak döner.

Arama ile ilgili üç kısayol bulunmaktadır. `Ctrl+F`'de kullanıcı yazdıkça arama yeniden yapılır ve tüm eşleşmeler anlık olarak mavi arka planla gösterilir; Enter ile onaylandığında imleç imlece en yakın sonraki eşleşmeye taşınır. `Ctrl+G`'de tüm eşleşmeler bulunduktan sonra kullanıcı sağ/aşağı yön tuşlarıyla sonraki, sol/yukarı tuşlarıyla önceki eşleşmeye geçer. Geçiş $i \leftarrow (i \pm 1) \bmod k$ ile yapıldığından sondan sonra başa dönülür; o an üzerinde durulan eşleşme açık mavi ile ayrıca vurgulanır. `Ctrl+H` ise bir sonraki alt bölümde açıklanmıştır.

Algoritma 3 Tümünü Değiştir

Girdi: Eski kelime e, yeni kelime y

```
M ← TümEşleşmeler(e)            ▷ (r, c)'ye göre sıralı
TopluİşlemBaşlat
her (r, c) ∈ M sondan başa yap
    Sil((r, c), (r, c + |e|))
    eğer y ≠ ε ise
        Ekle((r, c), y)
    koşul sonu
döngü sonu
TopluİşlemBitir                  ▷ tek Batch kaydı
döndür |M|
```

H. Toplu Değiştirme

`Ctrl+H` ile önce değiştirilecek kelime, ardından yeni kelime girilir; her iki aşamada da ESC işlemi iptal eder. Değiştirme sırasında yapılan bir silme işlemi, aynı satırdaki sonraki eşleşmelerin sütun numaralarını kaydırır. Bu sorunu ek hesaplama yapmadan çözmek için eşleşmeler sondan başa doğru işlenir: sondaki bir eşleşmenin değiştirilmesi, kendisinden önceki eşleşmelerin konumunu etkilemez (Algoritma 3). Tüm alt işlemler tek bir `BATCH` kaydında toplandığından, yüzlerce değişiklik tek bir `Ctrl+Z` ile geri alınabilir.

I. Dosya İşlemleri ve Dosya Yöneticisi

Dosya Aç (`Ctrl+O`), Dosya Kaydet (`Ctrl+S`) ve Dosya Farklı Kaydet (`Ctrl+Shift+S`) komutları konsol içinde çalışan bir dosya yöneticisi açar. Yönetici, mevcut dosya varsa onun bulunduğu dizinden, yoksa programın çalıştırıldığı dizinden başlar. Listenin başında üst dizini temsil eden ".." öğesi yer alır; ardından klasörler ve dosyalar büyük/küçük harf duyarsız alfabetik sırayla listelenir. Yukarı/aşağı tuşları öğeler arasında gezinmeyi, sağ ok (*ileri*) seçili klasörün içine girmeyi, sol ok (*geri*) ise üst dizine çıkmayı sağlar. Üst dizine çıkıldığında imleç az önce çıkılan klasörün üzerine konumlandırılır. Kullanıcı dosya adını yazar ve Enter ile işlemi tamamlar; ESC işlemi iptal eder. Açma modunda listede bir dosyanın üzerine gelindiğinde adı otomatik olarak giriş alanına yazılır. Kaydetme modunda var olan bir dosyanın adı girilirse üzerine yazma için onay istenir. Erişim izni olmayan klasörler hata mesajıyla bildirilir ve uygulama çökmez.

Proje tanımı gereği `Ctrl+S`, dosya daha önce kaydedilmişse dosya yöneticisini açmadan doğrudan aynı yola kaydeder. Dosyalar UTF-8 olarak kaydedilir. Açarken önce UTF-8 (BOM dahil) denenir; başarısız olursa Türkçe Windows kod sayfası olan CP-1254 kullanılır. Böylece eski Windows uygulamalarıyla oluşturulmuş Türkçe dosyalar da bozulmadan açılır.

`ESC` tuşu ile çıkılmak istendiğinde dosya kaydedilmemişse "Kaydetmek istiyor musunuz? (y/n)" sorusu sorulur. `y` yanıtında dosya kaydedilir ve çıkılır; dosyanın daha önce bir yolu yoksa dosya yöneticisi açılır, kullanıcı burada iptal ederse çıkış da iptal edilir. `n` yanıtında değişiklikler kaydedilmeden çıkılır. Arama vurgusu açıkken basılan ilk ESC yalnızca vurguyu kaldırır; bu sayede kullanıcı aramayı kapatmak isterken yanlışlıkla çıkış yapmaz. Benzer şekilde, değişiklik yapılmış bir dosya açıkken yeni bir dosya açılmak istenirse önce kaydetme sorusu sorulur.

J. Ekran Çizimi

Ekran yukarıdan aşağıya şu bölgelerden oluşur: toolbox şeridi, ayırıcı çizgi, satır numaralı metin alanı, durum çubuğu ve mesaj/giriş satırı. Her tuş işlendikten sonra tüm ekran tek bir karakter dizisi olarak oluşturulup tek bir `WriteConsoleW` çağrısıyla yazılır. Çizim süresince imlecin gizlenmesi ve her satırın kendi konumuna doğrudan yazılması titremeyi önler.

1) Toolbox şeridi: Her komut "Ad (Kısayol)" biçiminde bir parçaya dönüştürülür. Parçalar açgözlü (greedy) satır kırma yöntemiyle yan yana dizilir: bir sonraki parça mevcut satıra sığmıyorsa yeni bir satıra geçilir. Böylece proje tanımında istendiği gibi seçenekler aynı satırda yan yana dizilir ve sığmadığında ikinci satırdan devam eder. Pencere boyutu değiştiğinde şerit yeniden hesaplanır.

2) Görünüm penceresi: İmleç her zaman görünür olacak şekilde dikey (`top`) ve yatay (`left`) kaydırma değerleri güncellenir. Satır numarası sütununun genişliği satır sayısının basamak sayısına göre dinamik olarak hesaplanır.

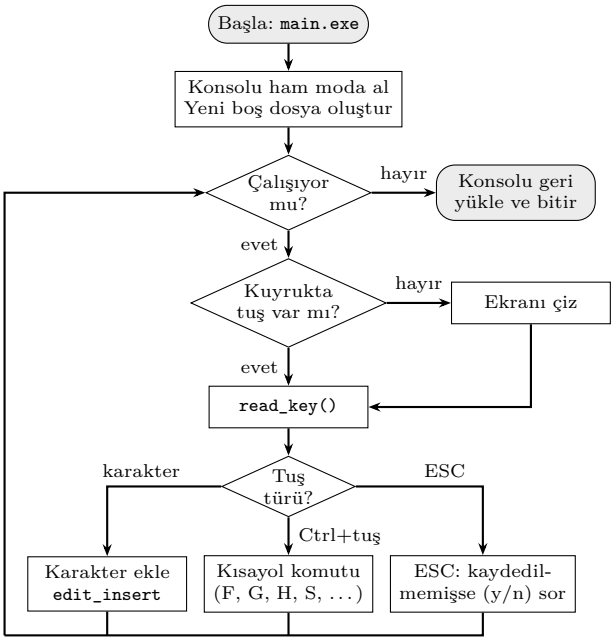
3) Renklendirme: Görünen her satır için karakter başına bir stil dizisi oluşturulur. Önce arama eşleşmeleri mavi, ardından seçim sarı olarak işaretlenir; seçim aramanın üzerine yazar. Ardışık aynı stildeki karakterler tek bir kaçış dizisiyle gruplanır. Seçim bir satırın sonunu kapsıyorsa, satır sonu karakterinin de seçildiğini göstermek için satırın sonuna sarı bir boşluk çizilir.

4) Ertelenmiş çizim: Ana döngü, çizimden önce `PeekConsoleInputW` ile kuyrukta işlenecek başka bir tuş olup olmadığını kontrol eder. Kuyrukta tuş varsa çizim atlanır. Bu sayede panodan büyük bir metin yapıştırıldığında ya da bir tuş basılı tutulduğunda yüzlerce ara kare yerine yalnızca son durum çizilir. Uygulamanın ana döngüsü Şekil 2 üzerinde gösterilmiştir.

III. Deneysel Sonuçlar

A. Test Yöntemi

Mimarideki bağımlılık enjeksiyonu sayesinde editör, gerçek Windows konsolu yerine önceden belirlenmiş bir tuş dizisini sırayla döndüren ve çizilen son ekran karesini kaydeden sahte bir konsol nesnesiyle çalıştırılabilmektedir. Bu yöntemle uygulama, uçtan uca kullanım senaryoları üzerinden otomatik olarak test edilmiştir. Her senaryo bir tuş dizisi ile başlar (örneğin "metin yaz, `Ctrl+H`'ye bas, eski kelimeyi yaz, Enter, yeni kelimeyi yaz, Enter, `Ctrl+Z`'ye bas") ve sonunda metin içeriği, imleç konumu, pano içeriği, diske yazılan dosyanın içeriği ya da ekran karesindeki renk kodları beklenen değerlerle karşılaştırılır. Platform katmanı ise Windows 11 klasik konsolunda (conhost) elle test edilmiştir. Başarım ölçümleri Python 3.12 ile x86-64 bir test ortamında, her ölçüm en az üç kez



Şekil 2. Uygulamanın ana döngüsüne ait akış diyagramı.

Tablo VI
Otomatik Test Senaryoları

Kategori	Doğrulanan davranışlar	Sonuç
Algoritma	LPS tablosu, KMP (çakışan eşleşmeler dahil)	2/2
Yazma ve silme	Türkçe karakter, Backspace, Ctrl+Backspace, satır birleştirme	3/3
Seçim ve pano	Karakter/kelime seçimi (iki yön), kopyala, kes, yapıştır, seçimin üzerine yazma, çok satırlı seçim	5/5
Geri alma	Kelime bazlı geri alma, tümünü geri alma, silme geri alma	3/3
Arama ve değiştirme	Ctrl+F vurgusu, ESC davranışı, Ctrl+G ileri/geri ve başa dönme, Ctrl+H, iki aşamada ESC ile iptal, tek adımda geri alma	8/8
Ekran	Satır numarası artışı/azalışı, toolbox, sarı seçim, dikey kaydırma	5/5
Dosya işlemleri	Yeni kaydetme, tekrar kaydetme, alt klasöre kaydetme, geri gitme, üzerine yazma onayı, açma	6/6
Çıkış	Temiz çıkış, (y/n) sorusu, `n`, `y` ile kaydedip çıkma	4/4
Toplam		**36/36**

tekrarlanıp en iyi süre alınarak yapılmıştır; ölçüm betiği (`bench.py`) proje dosyalarıyla birlikte verilmiştir.

B. Fonksiyonel Testler

Toplam 36 otomatik test senaryosunun tamamı başarıyla geçmiştir. Senaryoların kategorilere göre dağılımı Tablo VI üzerinde, proje tanımındaki gereksinimlerin karşılanma durumu ise Tablo VII üzerinde verilmiştir.

Tablo VII
Proje Gereksinimlerinin Karşılanma Durumu

Gereksinim	Gerçekleme
`main.exe` ile yeni boş dosya açılması	PyInstaller, `main.py`
Toolbox, kısayollar parantez içinde, taşma	Açgözlü satır kırma
Fare kullanılmaması	Fare girişi kapalı
Yanıp sönen beyaz dikey imleç	DECSCUSR + OSC 12
Türkçe karakter desteği	UTF-16 G/Ç, CP-1254
Dosya yöneticisi, ileri/geri, Enter	`filemanager.py`
Kayıtlı dosyada Ctrl+S'nin yönetici açmaması	`cmd_save`
Ctrl+F mavi vurgu	KMP + ANSI 44
Ctrl+G eşleşmeler arası gezinme	Modüler indeks
Ctrl+H tümünü değiştir, ESC iptal	Sondan başa, Batch
Ctrl+Yön / Ctrl+Shift+Yön seçimi, sarı	`Selection`, ANSI 103
Ctrl+C / X / V / Z	Pano + `Stack`
Backspace / Ctrl+Backspace	Karakter / kelime silme
Dinamik satır numaraları	Liste indeksi + 1
ESC ile çıkış, (y/n) sorusu	`cmd_exit`
Karakterlerin veri yapısında tutulması	Satır listesi
Undo için stack yapısı	Özgün `Stack` sınıfı
Seçimin indekslerle takibi	`start/end`
String eşleşme algoritması	KMP
Harici GUI kütüphanesi kullanılmaması	Yalnız standart kütüphane

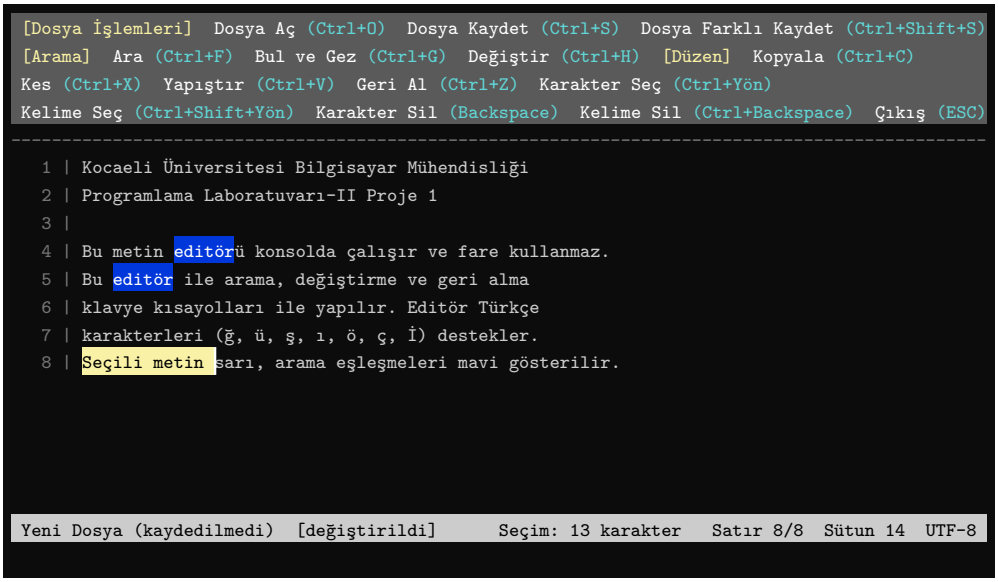
C. Ekran Görüntüleri

Şekil 3, 4 ve 5'de uygulamanın farklı durumlardaki ekranları verilmiştir. Bu görüntüler, editörün konsola gönderdiği karakter ve renk kodlarından birebir üretilmiştir. Şekil 3'de "editör" kelimesine ait arama eşleşmeleri mavi, `Ctrl+Shift+Sağ` ile seçilen iki kelime sarı arka planla görülmekte; durum çubuğu seçimin 13 karakter olduğunu göstermektedir.

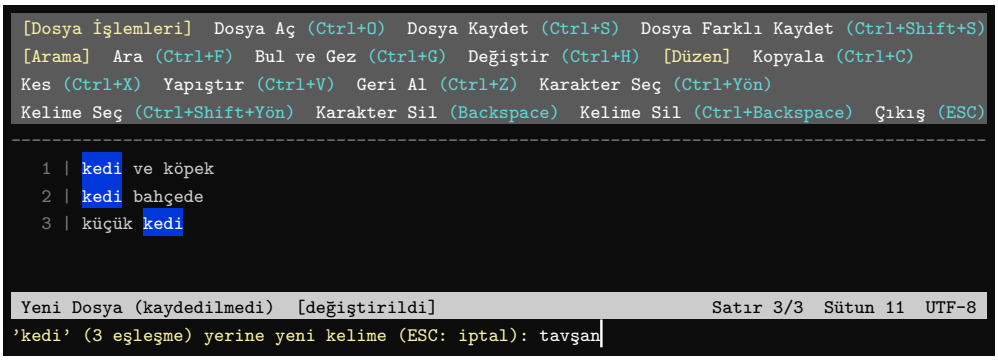
D. Başarım Ölçümleri

1) Ekleme maliyeti: Bölüm II'deki veri yapısı tercihini sınamak için dosyanın ortasına 1000 karakter eklenmiş ve işlem başına süre ölçülmüştür. Karşılaştırma için metnin tek parça bir string olarak tutulduğu yapı kullanılmıştır. Şekil 6'de görüldüğü gibi tek parça yapıda maliyet dosya boyutuyla doğrusal arttığı hâlde, satır listesi yapısında dosya boyutundan bağımsız olarak yaklaşık 3 μs'de sabit kalmaktadır. 50.000 satırlık (yaklaşık 4 MB) bir dosyada satır listesi yapısı yaklaşık 2400 kat daha hızlıdır.

2) Arama: KMP ile naif arama, naif algoritmanın en kötü durumunu oluşturan bir girdi üzerinde karşılaştırılmıştır: yalnızca "a" karakterlerinden oluşan bir metinde, 49 adet "a" ve bir "b"den oluşan ($m = 50$) kelime aranmıştır. Bu girdide naif algoritma her konumda 50 karakter karşılaştırıp başarısız olurken KMP metinde hiç geri dönmez. Sonuçlar Şekil 7'de verilmiştir: KMP her boyutta yaklaşık 18 kat daha hızlıdır ve her iki algoritmanın süresi



Şekil 3. Editörün ana ekranı: toolbox şeridi, satır numaraları, mavi arama eşleşmeleri, sarı seçim ve durum çubuğu.



Şekil 4. Ctrl+H ile değiştirme: aranan kelimenin eşleşmeleri vurgulanırken alt satırda yeni kelime isteniyor.

Tablo VIII
1000 Düzenleme İçin Geri Alma Geçmişinin Bellek Kullanımı

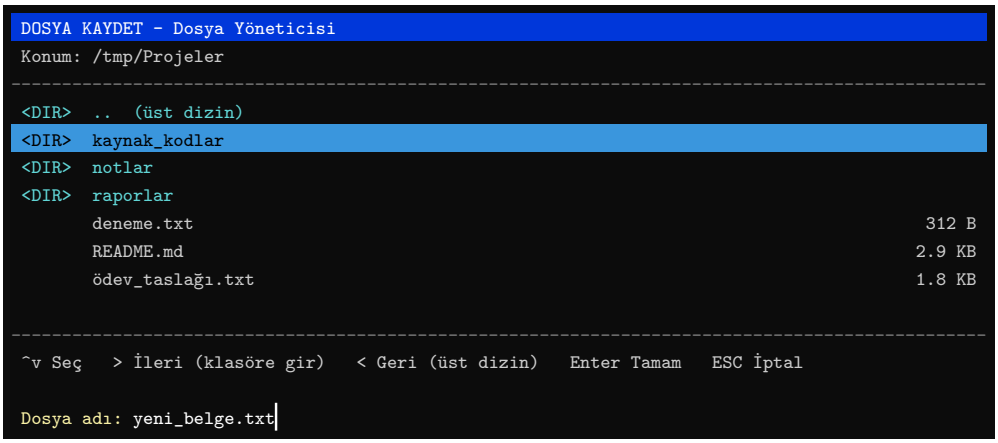
Dosya boyutu	İşlem kaydı	Anlık görüntü	Oran
8,0 KB	90,0 KB	16,11 MB	≈179×
81,2 KB	90,0 KB	162,55 MB	≈1806×
404,5 KB	90,0 KB	809,00 MB	≈8989×

de beklendiği gibi n ile doğrusal artmaktadır (naif algoritmanın çarpanı m'dir). Gerçekçi bir metinde (satır başına 10 kelime) `find_all` fonksiyonu 1.000 satırı 10,5 ms'de, 10.000 satırı (0,81 milyon karakter) 99 ms'de, 50.000 satırı (4,04 milyon karakter) ise 502 ms'de taramıştır.

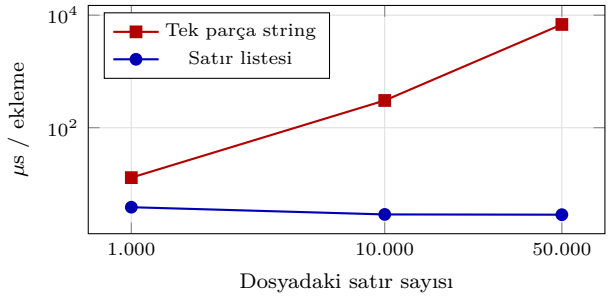
3) Geri alma bellek kullanımı: 1000 tek karakterlik düzenleme için işlem tabanlı yaklaşımın ve her düzenlemeden önce metnin kopyasını alan anlık görüntü yaklaşımının bellek kullanımları Tablo VIII üzerinde karşılaştırılmıştır. İşlem tabanlı yaklaşımın bellek kullanımı dosya boyutundan bağımsız olarak yaklaşık 90 KB'dir. Anlık görüntü yöntemi ise 404,5 KB'lık bir dosya için yaklaşık 809 MB bellek gerektirmektedir; bu, yaklaşık 9000 katlık bir farktır.

4) Toplu değiştirme ve geri alma: 1.000 satırlık bir metinde 918 eşleşme 18,2 ms'de değiştirilmiş ve tek bir `Ctrl+Z` ile 2,2 ms'de geri alınmıştır. 10.000 satır ve 9.058 eşleşme için bu süreler sırasıyla 243,4 ms ve 38,1 ms'dir. Her iki durumda da geri alma sonrası metnin orijinal metinle karakter karakter aynı olduğu doğrulanmıştır.

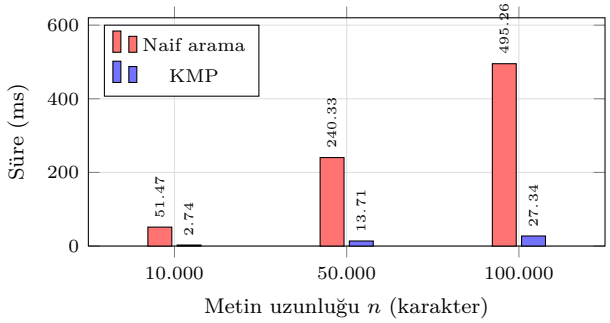
5) Ekran çizimi ve ölçüme dayalı iyileştirme: 120×30 boyutlu bir pencerede bir ekran karesinin çizimi ölçülmüştür. İlk ölçümlerde, arama vurgusu açıkken çizim süresinin dosya boyutuyla birlikte hızla arttığı görülmüştür: 50.000 satırlık bir dosyada kare başına 108,85 ms. İncelendiğinde, her karede ekranda görünmeyenler dahil 45.596 eşleşmenin tamamının tarandığı tespit edilmiştir. Eşleşme listesi (r, c) sırasına göre zaten sıralı olduğu için, yalnızca görünen satır aralığına düşen eşleşmeler ikili arama (`bisect`) ile $O(\log k)$ sürede bulunacak şekilde kod değiştirilmiştir. Tablo IX'de görüldüğü gibi bu iyileştirmeyle çizim süresi dosya boyutundan bağımsız hâle gelmiş ve en büyük dosyada yaklaşık 340 kat azalmıştır. Bir karenin çizimi 0,3 ms civarında sürdüğünden, kullanıcı tuşa bastığında gecikme algılanmamaktadır.



Şekil 5. Dosya Kaydet sırasında açılan konsol içi dosya yöneticisi: klasörler, dosya boyutları, kısayol yardımı ve dosya adı giriş alanı.



Şekil 6. Dosyanın ortasına karakter eklemenin işlem başına maliyeti (log–log ölçek).



Şekil 7. En kötü durum girdisinde ($m = 50$) naif arama ve KMP süreleri.

E. Karşılaşılan Sorunlar ve Çözümleri

Geliştirme ve test sürecinde karşılaşılan başlıca sorunlar şunlardır:

1) Terminal öykünücülerinin kısayolları yakalaması: Uygulama PyCharm geliştirme ortamının içinden çalıştırıldığında `Ctrl+H` (Type Hierarchy) ve `Ctrl+Backspace` gibi tuşların uygulamaya ulaşmadan geliştirme ortamı tarafından yakalandığı görülmüştür. Benzer şekilde Windows Terminal, `Ctrl+Shift+Yukarı/Aşağı` tuşlarını kendi kaydırma işlemi için kullanmaktadır. Çözüm olarak uygulamanın klasik Windows konsolunda (`conhost`) çalıştırılması belirlenmiştir.

2) Ctrl+H ve Backspace karışıklığı: ASCII'de `Ctrl+H` ile Backspace aynı koda sahip olduğu için bazı terminallerde `Ctrl+H` Backspace olarak algılanmıştır. Sorun, tuş çözümleme sırasında tarama kodu ve karakter değeri kontrol edilerek giderilmiştir (Tablo III, kural 1).

3) AltGr karakterleri: Türkçe klavyede `@` ve `{` gibi karakterlerin "Ctrl+Alt" olarak raporlanması nedeniyle bu karakterlerin kısayol olarak yorumlanma riski, Ctrl ile Alt birlikte basılıyken olayın karakter sayılmasıyla önlenmiştir.

4) Çalıştırılabilir dosya üretimi: Anaconda ortamında yüklü olan eski `pathlib` geri taşıma (backport) paketinin PyInstaller ile uyumsuz olduğu görülmüş; paket kaldırılarak sorun çözülmüştür. Derleme betiği de hata durumunda başarılı mesajı vermeyecek şekilde düzeltilmiştir.

Tablo IX
Arama Vurgusu Açıkken Kare Başına Çizim Süresi (ms)

Satır sayısı	Eşleşme	İyileştirme öncesi	Sonrası
100	≈90	0,31	0,34
10.000	≈9.000	11,97	0,30
50.000	45.596	108,85	0,32

IV. Sonuç

Bu çalışmada proje tanımındaki tüm gereksinimleri karşılayan, konsol tabanlı ve yalnızca klavyeyle yönetilen bir metin editörü geliştirilmiştir. Python standart kütüphanesi ile Windows konsol API'sine doğrudan erişilerek, harici bir kütüphane kullanılmadan ham klavye girişi, renkli vurgulama ve özelleştirilmiş imleç elde edilmiştir. Veri yapısı ve algoritma tercihleri hem kuramsal olarak gerekçelendirilmiş hem de deneysel olarak doğrulanmıştır: satır listesi yapısı ekleme maliyetini dosya boyutundan bağımsız hâle getirmiş, KMP algoritması en kötü durumda naif aramadan yaklaşık 18 kat hızlı çalışmış, işlem tabanlı geri alma ise bellek kullanımını binlerce kat azaltmıştır. Ölçümler sırasında tespit edilen bir çizim darboğazı

ikili arama ile giderilerek çizim süresi yaklaşık 340 kat iyileştirilmiştir. Bağımlılık enjeksiyonuna dayalı mimari sayesinde uygulamanın tamamı 36 uçtan uca senaryoyla otomatik olarak test edilebilmiştir.

Uygulamanın bazı sınırlamaları bulunmaktadır. Sekme karakterleri dört boşluğa dönüştürülmekte, ekrandaki her karakterin tek hücre genişliğinde olduğu varsayılmaktadır; bu nedenle emoji gibi çift genişlikli karakterler hizalamayı bozabilir. İleri alma (redo) desteği yoktur. Çok büyük dosyalarda (milyonlarca karakter) `Ctrl+F` sırasında her tuşta yeniden yapılan arama yarım saniyeye ulaşabilmektedir. Platform katmanı yalnızca Windows için yazılmıştır.

Gelecek çalışmalarda ikinci bir yığın ile ileri alma desteğinin eklenmesi, çok büyük dosyalar için piece table veya rope yapısına geçilmesi, aramanın yalnızca değişen satırlarda tekrarlanması, Boyer-Moore gibi alt-doğrusal ortalama süreli algoritmaların ve düzenli ifadelerin (regex) desteklenmesi, sözdizimi renklendirmesi ve Linux/macOS desteği için POSIX `termios` tabanlı ikinci bir platform katmanı yazılması planlanmaktadır. Katmanlı mimari sayesinde son madde, üst katmanlarda değişiklik gerektirmeden yapılabilecektir.

Kaynaklar

[1] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[3] C. A. Finseth, *The Craft of Text Editing: Emacs for the Modern World*. New York, NY, USA: Springer-Verlag, 1991.

[4] C. Crowley, "Data structures for text sequences," Computer Science Department, University of New Mexico, Tech. Rep., 1998.

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[6] ECMA International, "Control functions for coded character sets," Standard ECMA-48, 5th ed., 1991.

[7] Microsoft, "Console virtual terminal sequences," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/console/console-virtual-terminal-sequences

[8] Microsoft, "ReadConsoleInput function," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/console/readconsoleinput

[9] Microsoft, "SetConsoleMode function," *Microsoft Learn*. [Online]. Available: https://learn.microsoft.com/en-us/windows/console/setconsolemode

[10] Python Software Foundation, "ctypes — A foreign function library for Python," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/ctypes.html

[11] PyInstaller Development Team, "PyInstaller manual." [Online]. Available: https://pyinstaller.org/en/stable/

[12] D. Ho, "Notepad++: A free source code editor." [Online]. Available: https://notepad-plus-plus.org/